

From Workflow to Agent

ADLC HR Onboarding Demo — a working portal where an AI-style agent reasons about each new hire, retrieves the right policy, adapts its plan, and flags compliance.

Lightning Talk Companion

"How HR Onboarding Changes When AI Thinks"

Prepared: June 24, 2026

Version: 1.0

React 18 · Vite

Zero-dependency Java (JDK HttpServer)

EmailJS · SheetJS · Google Identity

Offline rule-based agent

Table of Contents

1. Project Overview & Setup Guide

- What's Inside — Repository Layout
- Sample Roster & Agent Branching
- Run It (≈2 Minutes)
- How It Works — Backend, Agent & Frontend
- Authentication (Demo-Only)
- Email: Mock vs Live (EmailJS)
- The Teaching Point — Workflow vs Agent
- Notes & Troubleshooting

2. Stage Cue Sheet — Presentation Guide

- Pre-Talk Setup
- Hook & SDLC vs ADLC Framing
- Demo — The Easy Case
- Demo — The Agent Earns Its Keep
- Key Lessons & Call to Action
- Backup / Q&A Pocket Answers

3. Application Walkthrough & Screenshots

- Startup Workflow (Run It)
- End-to-End User Workflow — with UI screenshots
- What the Onboarding Agent Does Per Hire
- Feature Summary

Project Overview & Setup Guide

From Workflow to Agent — ADLC HR Onboarding Demo

A live demo for the lightning talk **"From Workflow to Agent: How HR Onboarding Changes When AI Thinks."** It shows what onboarding looks like when a classic static **SDLC** workflow is replaced by an **ADLC** onboarding *agent* that reasons about each hire, retrieves the right policy, adapts its plan, retries on failure, and flags compliance — all from the same goal, with no new code branches.

The demo ships the *agent* side of that story as a working HR portal: import a roster (or add hires by hand), pick who to onboard, and watch the agent generate a personalised plan and welcome email for each one. It's a custom **React + EmailJS** frontend over a **zero-dependency Java** backend, so it runs anywhere with just Node and a JDK — no Flowise install, no cloud, no API keys required on stage.

The "before" (the rigid SDLC workflow) lives in the talk narrative; this app embodies the "after." See **TALK.md** for the full stage cue sheet.

What's inside

HR Portal_ADLC/

- backend/
 - src/com/cognizant/adlc/
 - Server.java
 - AdlcAgent.java
 - PolicyStore.java
 - Dsl.java / Json.java
 - onboarded-candidates.csv
 - removed-candidates.csv
 - run.ps1 / run.sh
- frontend/
 - src/
 - App.jsx
 - api.js
 - auth.js
 - email.js
 - parseHires.js
 - assistant.js
 - components/
 - .env.example
- policies/
- sample-new-hires.csv
- TALK.md
- README.md

Zero-dependency Java REST API (JDK HttpServer)

HTTP routing, CORS, CSV history endpoints

the onboarding agent (reasoning + tool calls)

simulated RAG over the policy docs

tiny helpers (no JSON library, no deps)

onboarding history (written at runtime)

removed-candidate audit trail

compile + run with a stock JDK

React (Vite) + EmailJS

portal shell: import/add → select → onboard

thin client for the Java backend

client-side session (email/password or Google)

EmailJS (mock by default, live via .env)

CSV/XLSX import + field normalisation

offline NL assistant (no LLM, no network)

login, import panel, hire form, results, dashboard, assistant panel, theme toggle...

backend URL, Google client ID, EmailJS config

mock policy "PDFs" the RAG retrieves from

a sample roster you can import

the 15-20 min lightning-talk cue sheet

The bundled `sample-new-hires.csv` is a global roster (FTEs, contractors and interns across NA, EU, APAC, ASIA and LATAM) chosen to exercise every branch the agent reasons over:

Dimension	What the agent does with it
Employment type (FTE / contractor / intern)	Picks which tools apply — buddy, managed laptop, payroll & benefits vs. BYOD + scoped VPN vs. stipend (no payroll)
Region / country (esp. EU)	Retrieves the governing policy; raises GDPR + works-council flags for EU hires
Start timing (quarter-start / mid-quarter / quarter-end)	Live cohort vs. self-paced orientation + manager 1:1 / reserved next-quarter seat
Locale (greeting)	Localises the welcome email (e.g. "Hallo", "Namaste", "Bonjour")

Edit the CSV, or add a hire from the form, to demo any combination on stage.

Run it (≈2 minutes)

You need **Node 18+** and a **JDK 17+** (built and tested on Node 25 / JDK 25). No Maven, no Gradle.

1. Start the backend (terminal 1)

```
cd backend
./run.ps1 # compiles src/ to out/, serves http://localhost:8080
```

On macOS/Linux: `cd backend && ./run.sh`

Sanity check: open `http://localhost:8080/api/health` → `{"status":"ok",...}`

2. Start the frontend (terminal 2)

```
cd frontend
npm install      # first time only
npm run dev      # opens http://localhost:5173
```

3. Drive the demo

1. **Sign in.** Any valid email + a 4-character password works (auth is local to the demo), or use the Google button. There's no auth backend — see *Auth* below.
2. **Choose who to onboard** on the **Onboarding** tab: - **Import file** — drop a `.csv / .xlsx` (try `sample-new-hires.csv`); rows load into a candidate table you can search, select, edit and trim, or - **Add a new hire** — fill the form to append one candidate.
3. **Select** the candidates you want and click ► **Onboard**. The agent generates a plan for each, then the welcome emails are sent (mock by default).
4. Open the **Dashboard** tab for analytics over everything onboarded so far, or use the floating **AI assistant** (bottom-right) to drive the portal by typing plain English — e.g. *"import the sample, onboard all interns in Asia."*

How it works

Backend — a tiny REST API (no frameworks)

`Server.java` runs on the JDK's `com.sun.net.httpserver` and exposes:

Route	Purpose
GET <code>/api/health</code>	Liveness probe (the UI shows <i>service online/offline</i>)
POST <code>/api/onboard</code>	Takes one new-hire JSON object → returns its personalised onboarding plan
GET/POST <code>/api/onboarded</code>	Onboarding history CSV — GET reads it, POST appends a sent invitation
GET/POST <code>/api/removed</code>	Removed-candidate audit trail CSV
GET/POST <code>/api/candidates</code>	The working-roster store — POST persists the edited roster (and mirrors it to the sample)
GET <code>/api/sample</code>	The curated <code>sample-new-hires.csv</code> — what the assistant's "import sample" loads (with your saved edits)

History and the working roster are persisted as plain CSV files in `backend/` (`onboarded-candidates.csv` , `removed-candidates.csv` , `candidates.csv`), so the dashboard and the candidate table survive restarts.

The agent — `AdlcAgent.java`

For each hire the agent emits the *shape* of agentic output — reasoning, tool calls, reflections and compliance flags — by:

1. **Perceiving** the profile into working memory (`hr_lookup`),
2. **Classifying** by employment type / region / department / timing,
3. **Retrieving policy via RAG** (`PolicyStore.java`) — the right policy doc for *this* profile, with citations,
4. **Drafting a localised welcome email**,
5. **Inviting to Slack** — the first call returns a simulated HTTP 429, so the agent reflects and retries instead of failing the run,
6. **Branching on employment type** (contractor → BYOD/liaison, no FTE benefits; intern → buddy + laptop, stipend not payroll; FTE → buddy + laptop + payroll),
7. **Raising compliance flags** for EU hires (GDPR Art. 13 + works council),
8. **Scheduling orientation** based on start timing.

The reasoning is deterministic, rule-based logic dressed as agent "thoughts" — chosen so the live demo never flakes. The output shape matches a real LLM-driven agent, so wiring in the Anthropic API later is a drop-in at `AdlcAgent.java` . The `policies/` markdown files are the mock "PDFs" the RAG retrieves from — open one on stage to show where a citation comes from.

Frontend — the portal (`App.jsx`)

- **Import / add hires** — `parseHires.js` reads CSV/XLSX with forgiving header aliasing and normalises values to the enums the agent expects. Imported rows become a selectable candidate table; edit an email inline or click the 📧 icon to edit a candidate's full details. Every change is saved to the backend and mirrored into `sample-new-hires.csv` (non-empty rosters only — clearing the table never wipes the sample). The table starts empty on load; use the assistant's "import sample" or the Import tab to (re)load the saved `sample-new-hires.csv` with your edits.
- **Batch onboarding** — each selected hire is POSTed to `/api/onboard` ; the results render step-by-step, then welcome emails are bulk-sent. The run button locks afterwards so a batch can't be onboarded twice.
- **Dashboard** — pulls the persisted history and shows counts, breakdowns (by region / type / department) and searchable tables.
- **Offline AI assistant** (`assistant.js`) — a deterministic, regex-based "agent" with **no LLM and no network call**. It parses plain-English

- commands into a plan of tool calls, runs them against the live portal, and narrates the result — so the assistant demo works on any machine, offline, behind any proxy.
- **Theme toggle** — light / dark / system.

Auth (demo-only)

There is no auth backend in this prototype:

- **Email/password** is validated locally (any valid email + 4-character password) and the session is kept in `localStorage` so a refresh stays signed in.
- **Google sign-in** uses real Google Identity Services when `VITE_GOOOGLE_CLIENT_ID` is set; otherwise a clearly-labelled demo Google account is used so the flow still reaches the portal on stage.

Email: mock vs live (EmailJS)

The welcome email is generated by the backend and rendered in the UI. Sending is handled client-side by **EmailJS**.

- **Mock mode (default)** — `VITE_EMAILJS_MODE=mock` . Sending is simulated and a confirmation is shown. No account, no network. **Use this on stage.**
- **Live mode** — set in `frontend/.env` :

```
VITE_EMAILJS_MODE=live
VITE_EMAILJS_SERVICE_ID=your_service_id
VITE_EMAILJS_TEMPLATE_ID=your_template_id
VITE_EMAILJS_PUBLIC_KEY=your_public_key
```

Your EmailJS template should use: `{{name}}` in the **Subject**, `{{to_email}}` as the **To Email**, and `{{message}}` followed by `{{footer}}` in the **Content** (body + footer note). Restart `npm run dev` after editing `.env` .

See `.env.example` for every setting (backend URL, Google client ID, EmailJS keys).

What changes when the workflow becomes an agent (the teaching point)

Static SDLC workflow (the "before")		ADLC agent (this demo)
Control flow	Fixed steps, same for everyone	Plans the tools per hire from one goal
New hire type	Needs a new code branch	Reasons about it at runtime
Policy	Hard-coded assumptions	Retrieved via RAG (with citations)
Failure	Step fails / wrong output	Reflects → retries → escalates
Compliance	A blind spot (no step for it)	Flagged for a human, with source
Changing it	New release	New prompt / tool — no redeploy

For a **Germany contractor**, a static workflow would still run its fixed steps — getting the buddy, IT checklist and benefits *wrong* while silently missing GDPR and works-council obligations. The agent instead retrieves `DE-Contractor-Policy.pdf` , skips the inapplicable steps, provisions BYOD, raises the GDPR flag, and localises the email to "Hallo" — with no new code.

Notes & troubleshooting

- **"service offline" banner** → the backend isn't running or is on another port. Start `backend/run.ps1` ; if you change the port, set `VITE_API_BASE` in `frontend/.env` .
- **Port already in use** → `./run.ps1 9090` to pick another backend port (then update `VITE_API_BASE`).
- **Recompile backend after editing Java** → just re-run `run.ps1` (it always recompiles `src/` into `out/`).
- **Dashboard is empty** → onboard a few candidates first; history is written to the CSV files in `backend/` as invitations are sent.

Stage Cue Sheet — Presentation Guide

Stage Cue Sheet — "From Workflow to Agent"

Lightning Talk · 15-20 min · mixed audience. Cues map the talk structure to this demo: what to *click* and what to *say*. Times are cumulative-ish targets.

Before you walk on: backend running (`backend/run.ps1`), `npm run dev` up, browser on `http://localhost:5173` , **signed in** (any email + a 4-character password, or the Google button), on the **Onboarding** tab. Header shows ● **service online** and **email: mock**. Optionally pre-import `sample-new-hires.csv` so the roster is already on screen. Zoom the browser to ~110-125% for the room.

This build ships the **agent**. The rigid SDLC workflow is the *conceptual* contrast (the §2 slide) — you narrate what a hard-coded workflow would do; you don't click through one. That's the point: there's no five-branch workflow left to maintain.

1 · Hook (2 min) — *no clicks yet*

"A new hire joins. 11 tools, 6 forms, 4 departments, zero context. What happens? A human runs around connecting the dots. Today I want to show you what happens when the *system* connects the dots instead — when it can **think**."

Set the frame: onboarding is the perfect lab — multi-step, context-sensitive, full of exceptions. We'll look at it through two lenses.

2 · SDLC vs ADLC, conceptually (4 min) — *slide*

Land the contrast (slide):

SDLC	ADLC
Requirements → Design → Build → Test → Deploy	Goal → Agent design → Tool binding → Eval → Deploy + monitor
Deterministic steps	Reasoning + dynamic tool use
Human handles exceptions	Agent handles exceptions
Linear, versioned releases	Iterative, continuous tuning

"SDLC builds a **conveyor belt**: five fixed steps, the same for everyone, and a human patches every exception. ADLC builds a **thinking co-worker** that plans the steps per hire. Let me show you the co-worker." → switch to the browser: one portal, one agent.

3 · Demo — the easy case (3 min)

- Import the roster.** *Import file* → choose `sample-new-hires.csv` . The candidate table fills — ~18 hires, mixed FTE / contractor / intern across NA, EU, APAC, ASIA and LATAM. "*A real-ish roster: different types, countries, start times.*"
- Onboard one standard full-timer.** Clear the *select-all* checkbox, then tick just **Sarah Johnson** (Consultant · FTE · US · quarter-start). Click ► **Onboard 1 candidate**.
- Watch the plan stream in: profile loaded → policies retrieved → welcome email → Slack invite → buddy → managed laptop → payroll & benefits → live cohort. All green ✓ , **0 compliance, 0 skipped**. Expand "**How the agent planned this onboarding**" to show the reasoning behind it.

"For a standard full-timer this is exactly what you'd expect — and honestly a hard-coded workflow would nail it too. This is the demo everyone gives. The difference shows up on the **exceptions**. So let's hire a contractor. In Germany. Watch the *same* agent — with no new code."

4 · Demo — the agent earns its keep (5 min)

Tick **Tom Becker** (DevOps Engineer · **Contractor** · **Germany / EU** · quarter-start). *Selecting a new candidate re-arms the Onboard button* after the previous run. Click ► **Onboard 1 candidate** and walk the result top to bottom.

- Header stats** — ~8 actions, **2 compliance, 2 skipped**. Already a different plan from Sarah's.
- Onboarding plan** — point out it's **not** the same steps: - **Profile loaded** (`hr_lookup`) → `CONTRACTOR · Germany` . - **Policies retrieved** (`policy_rag`) → expand the doc snippets: `DE-Contractor-Policy.pdf §2 + §4` and `IT-Access-Guide.pdf §1`, with citations. "*This is RAG — it pulled the right policy for this exact hire.*" - **Slack workspace invite** → **429 rate-limited** ⚠ , then "**sent on retry.**" "*It reflected on a transient failure instead of dying.*" - **IT provisioning** (`itsm`) → BYOD + scoped VPN — *not* a managed laptop. - **HR system** (`hris`) → a contractor liaison — *not* an FTE buddy.

3. **Compliance & approvals** — **GDPR Art. 13** (warning) + **Betriebsrat / works-council** (info), each with a source. *"It flagged what a human must see — the blind spot a five-step workflow has no box for."*
4. **Not applicable for this hire** — FTE payroll & benefits, and the FTE buddy (replaced by the liaison). *"It skipped those on purpose."*
5. **Policies applied** — `DE-Contractor-Policy.pdf` , `IT-Access-Guide.pdf` .
6. **Welcome email** — **"Hallo Tom"**, contractor-aware (BYOD, no payroll/buddy), with the GDPR acknowledgement line. The email **auto-sent in mock mode** — the green banner confirms "*✓ Mock-sent 1 welcome email.*" (Wire EmailJS live in `.env` and it really sends.)

"Same goal. Different hire. **Zero new code.** It reasoned, retrieved, adapted, recovered, and escalated. That's the difference between a workflow and an agent."

(Optional 20s flexes:)

- **Timing adapts too:** onboard **Carlos Mendoza** (mid-quarter FTE, Brazil) — the agent swaps the live cohort for **self-paced orientation + a manager 1:1**.
- **It's all tracked:** open the **Dashboard** tab — counts and breakdowns by region / type / department across everyone you just onboarded.
- **Hands-free:** open the floating **AI assistant** and type *"onboard all interns in Asia"* — a no-API, offline assistant drives the portal from plain English.

5 · Key lessons (3 min) — *back to slide*

- ADLC **extends** SDLC; it doesn't replace it — reach for it on ambiguous, multi-step, context-sensitive problems.
- The lifecycle shift: **evaluation is continuous** (prompt drift, tool failures, hallucination), not a one-time QA gate.
- **Low-code makes it accessible** — you don't need to be an ML engineer. (This demo is a React app + a small zero-dependency Java backend + simulated reasoning.)
- **New roles emerge:** Agent Designer, Prompt Engineer, Evaluation Ops.

6 · Call to action (1 min)

- **Developers:** rebuild one brittle workflow as an agent this week.
- **Non-devs:** the agent you saw is a config, not a PhD — open the tools today.
- **Everyone:** *"Where in your org does a human run around connecting dots?"* That's your first agent.

Backup / Q&A pocket answers

- **"Where's the SDLC version in the app?"** — Deliberately not here. The rigid five-step workflow is the conceptual contrast on the slide; the whole point is you stop maintaining hard-coded branches. The agent *is* the after.
- **"Is it really thinking / a real LLM?"** — In this build the reasoning is deterministic so the demo can't flake on stage; the *architecture* (goal → reason → dynamic tools → reflect → flag) is exactly an LLM agent's, and `AdlcAgent.java` is a drop-in for the Anthropic API.
- **"What's the RAG?"** — The `policies/` folder holds the mock policy docs; `policy_rag` retrieves the sections relevant to the hire (e.g. `DE-Contractor-Policy.pdf` for a German contractor) and returns citations.
- **Onboard button greyed out / says "✓ Onboarded"** — that's the double-run guard. Tick a different candidate (or import/add one) to re-arm it.
- **If the network dies** — everything runs locally; email is in **mock** mode by default. Nothing on stage needs the internet.
- **If the backend banner is red** ("service offline") — restart `backend/run.ps1` ; check the port matches `VITE_API_BASE` in `frontend/.env` .

Application Walkthrough & Screenshots

Application Walkthrough & Screenshots

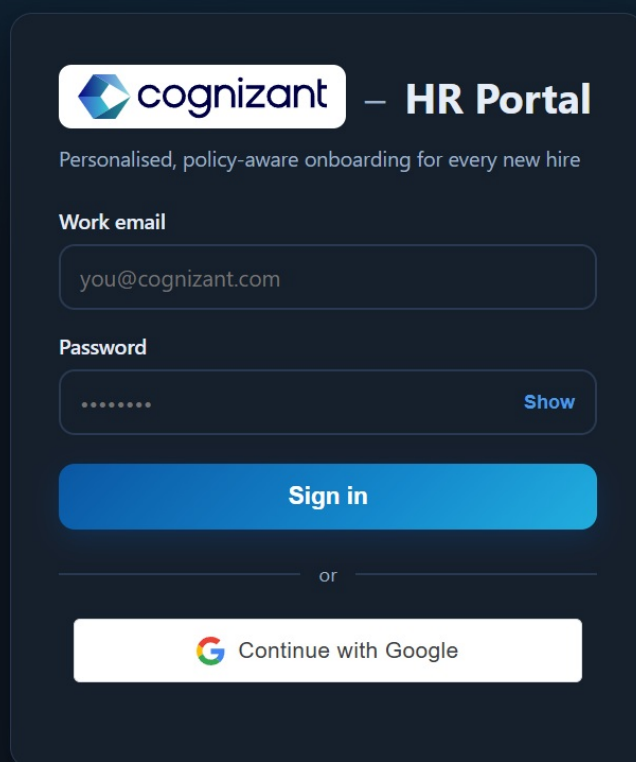
This section captures the application in its current running state: the end-to-end operator workflow, what the onboarding agent does for each hire, and the full feature set — illustrated with screenshots of the live portal in both light and dark themes.

A • Startup Workflow (Run It)

1. **Start the backend** — `cd backend; ./run.ps1` → compiles the Java sources and serves on `http://localhost:8080`.
2. **Start the frontend** — `cd frontend; npm install` (first time) ; `npm run dev` → Vite dev server on `http://localhost:5173`.
3. The frontend reads `VITE_API_BASE` from `.env` to reach the backend; on load it pings `GET /api/health` and shows *service online/offline* in the menu.

B • End-to-End User Workflow

- 1 • **Sign in** (`Login.jsx`) — email + password (any valid email, 4+ char password) or Google. Session is saved in `localStorage`. Login has no theme button; it shows whatever theme was chosen in the menu.



The screenshot shows the Cognizant HR Portal login interface. At the top, the Cognizant logo is followed by the text 'HR Portal'. Below this is the tagline 'Personalised, policy-aware onboarding for every new hire'. The form has two main sections: 'Work email' and 'Password'. The 'Work email' field contains the text 'you@cognizant.com'. The 'Password' field is masked with dots and has a 'Show' button to its right. Below these fields is a large blue 'Sign in' button. Underneath the button is a horizontal line with the word 'or' in the center. At the bottom is a white button with the Google logo and the text 'Continue with Google'.

Fig. 1 Login page with email/password and Google OAuth 2.0 sign-in

2 · Land on the Onboarding page — the header shows the brand; the nav row has ☰ menu · Onboarding · Dashboard. The candidate table starts empty (nothing auto-loads).



Fig. 2 Onboarding page on first load (dark mode) — empty state with CSV/XLSX drop zone

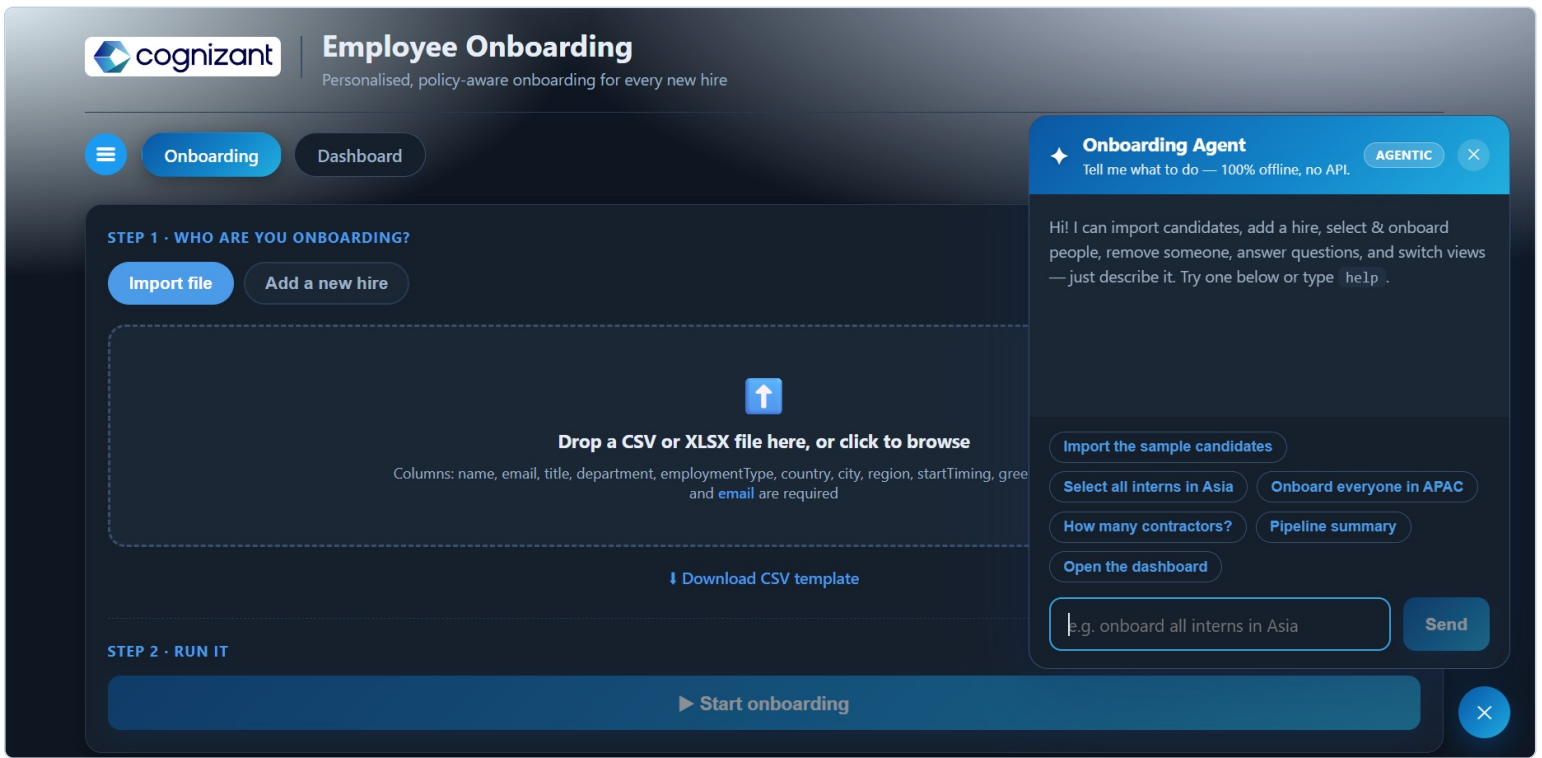


Fig. 3 Onboarding page with the floating Onboarding Agent open — 100% offline, no API

3 • Open the side menu (☰) — pushes the page aside (desktop) / overlays it (mobile). Holds profile details, screen theme (Light/Dark/System), service status, and Sign out. Theme is controlled only here and applies app-wide (including login).



Fig. 4 Side menu open — profile, screen theme toggle, service status and sign-out

4 • Add candidates (Step 1) — either *Import file* (drop a CSV/XLSX), *Add a new hire* (fill the form), or tell the agent “import the sample” to load the saved `sample-new-hires.csv`.

STEP 1 • WHO ARE YOU ONBOARDING?

Import file

Add a new hire

Full name *

e.g. Asha Rao

Work email *

asha.rao@cognizant.com

Job title

e.g. Software Engineer

Department

Select department...

City

e.g. Chennai

Country

e.g. India

Region

North America

Local greeting

Dear

Employment type

Full-time

Contractor

Intern

Start timing

Quarter start

Mid-quarter

Quarter end

Used to localise the welcome email

STEP 2 • ADD TO THE CANDIDATE LIST • 13 candidates ready

+ Add candidate

Ask

Fig. 5 Add-a-new-hire form — name, email, title, department, region, greeting, type and start timing

cognizant

Employee Onboarding

Personalised, policy-aware onboarding for every new hire

Onboarding

Dashboard

STEP 1 • WHO ARE YOU ONBOARDING?

Import file

Add a new hire

sample-new-hires.csv (via assistant)

13 of 13 rows imported

Tip: click the icon to edit a candidate's details · use the search row to filter columns.

CANDIDATE	TITLE	DEPARTMENT	TYPE	REGION
☒ Saji sajiad674@gmail.com	HR	Human Resources	Full-time	APAC
☒ Diego Fernandez mohitsant2004@gmail.com	Account Executive	Sales	Full-time	LATAM
☒ Tom Becker mohitsant2004@gmail.com	DevOps Engineer	Engineering	Contractor	EU
☒ Sarah Johnson mohitsant2004@gmail.com	Consultant	Consulting	Full-time	NA

Onboarding Agent

Tell me what to do — 100% offline, no API.

AGENTIC

×

Import the sample candidates

PLAN Import sample candidates

✓ Import sample candidates

13 loaded

Imported 13 candidates from the sample file and selected them all.

Import the sample candidates

Select all interns in Asia

Onboard everyone in APAC

How many contractors?

Pipeline summary

Open the dashboard

e.g. onboard all interns in Asia

Send

Quarter start

Remove

Quarter start

Remove

Fig. 6 Candidates imported via the agent — 13 rows loaded and selected automatically

5 • Manage the candidate table — select who to onboard, click the icon to edit a candidate’s full details in a modal, or remove someone. The email shows read-only (edit it via the modal). Every change is saved to `candidates.csv` and mirrored to `sample-new-hires.csv`.

6 • Run onboarding (Step 2) — for each selected hire the frontend calls `POST /api/onboard` ; the agent builds a personalised plan (see section C), and the result renders step-by-step.

7 • Welcome emails sent — after plans generate, emails are bulk-sent (mock by default; live via EmailJS), and each sent candidate is logged to `onboarded-candidates.csv` .

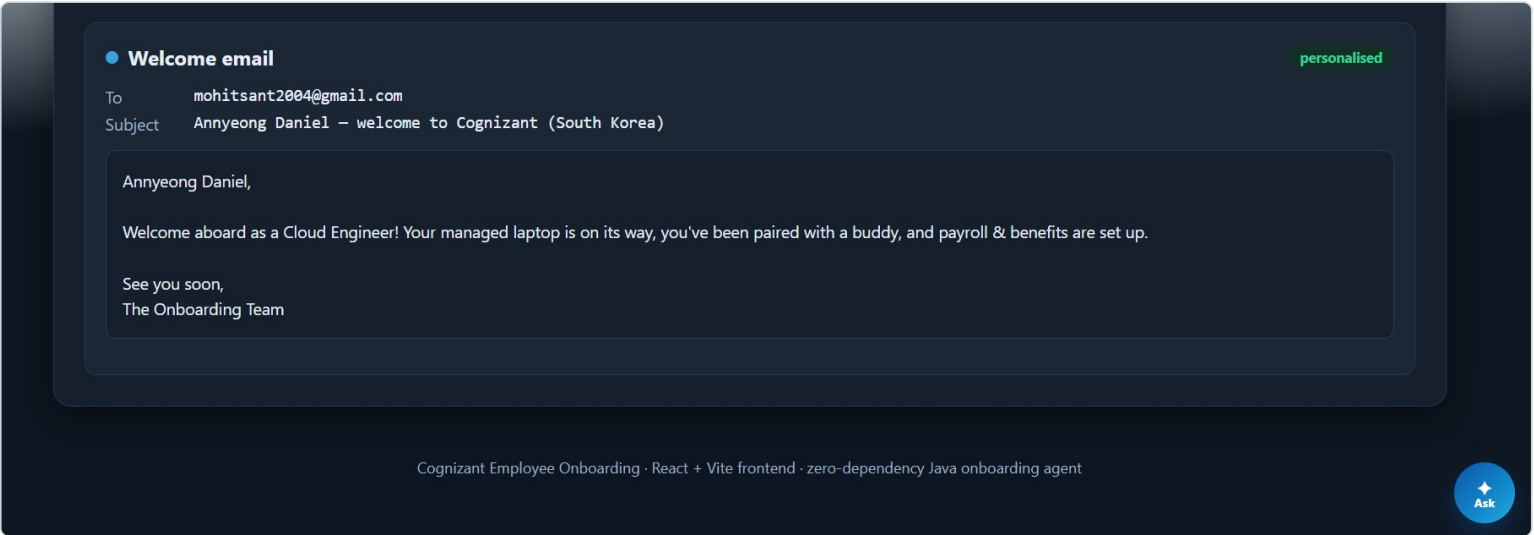


Fig. 7 Personalised welcome email generated by the backend and sent via EmailJS (localised greeting)

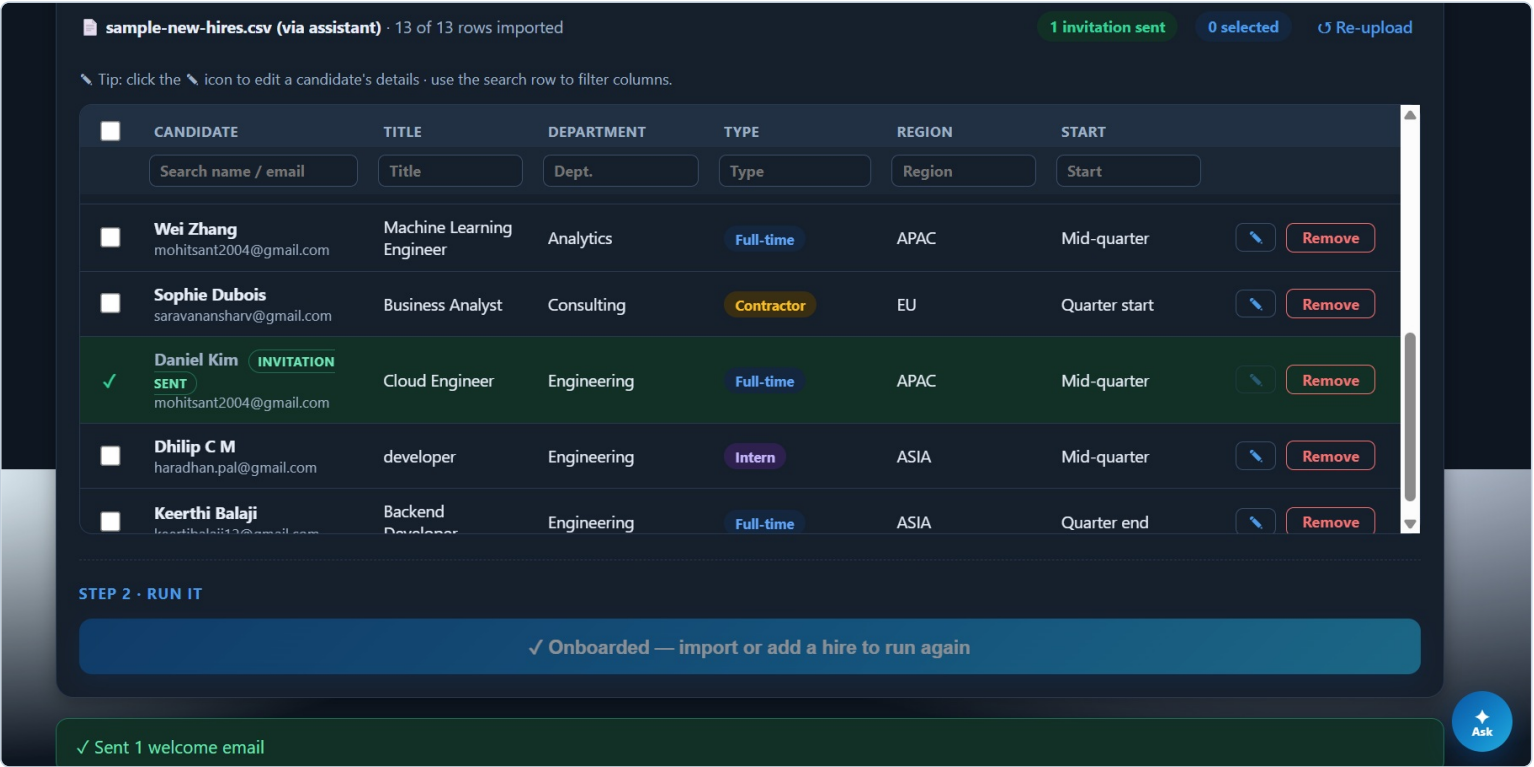


Fig. 8 Real-time confirmation — invitation sent for the selected candidate, logged to history

8 • Review on the Dashboard — switch to the Dashboard tab for analytics over everyone onboarded/removed (counts, breakdowns, searchable history) pulled from the backend CSVs.

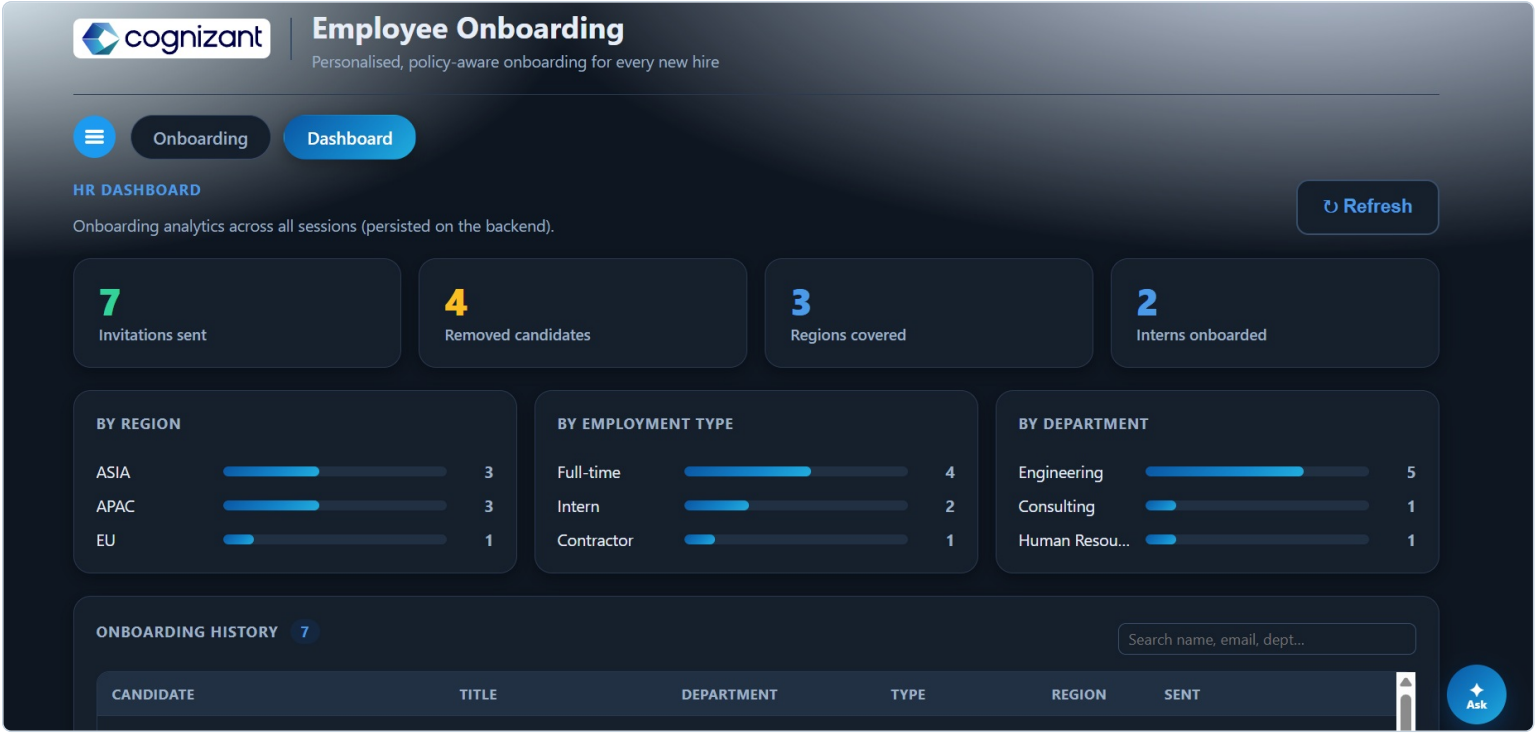


Fig. 9 HR Dashboard — invitations, removed candidates, breakdowns by region/type/department and onboarding history

9 • Drive it hands-free via the Onboarding Agent — the floating chatbot (bottom-right) takes plain-English commands at any point.

10 • Sign out — from the side menu; returns to login (theme persists).



Fig. 10 Agent executing a task inside the portal — “Summarise the pipeline” with live counts by type and region

C • What the Onboarding Agent Does Per Hire (AdLcAgent.java)

1. Loads the profile (perceive / memory).
2. Classifies the hire (employment type, region, department, timing).
3. Retrieves the right policy via RAG (PolicyStore.java) with citations.
4. Drafts a localised welcome email.
5. Invites to Slack — simulates a transient failure, then reflects & retries.
6. Adapts the plan — contractor → BYOD + liaison (no buddy/payroll); intern → buddy + laptop (stipend, no payroll); FTE → buddy + laptop + payroll.
7. Raises compliance flags — e.g. EU → GDPR + works-council, with source.

8. Schedules orientation by start timing (live cohort vs. self-paced + 1:1).

Logic is deterministic / simulated — no real LLM — so the demo never flakes.

D · Feature Summary

- **Auth:** local email/password + Google sign-in (demo).
- **Onboarding:** CSV/XLSX import or manual add; selectable candidate table; full-detail edit modal; remove.
- **AI-style agent:** reasoning + dynamic tool calls + policy RAG + compliance flags + retry, per hire.
- **Welcome emails:** EmailJS, mock or live.
- **Offline Onboarding Agent chatbot:** import / select / onboard / remove / list / count / stats / theme / view — all via plain English, no LLM/network.
- **Dashboard:** persisted onboarding/removed analytics.
- **Side menu:** profile, theme (Light/Dark/System), status, sign-out; push-aside on desktop, overlay on mobile.
- **Persistence:** CSV files only (`candidates.csv` , `onboarded-candidates.csv` , `removed-candidates.csv` , mirrored `sample-new-hires.csv`) — no database.
- **Theming:** light/dark/system, controlled solely from the menu, applied app-wide.